Project: Beyond the Classic TSP

Redesigning Algorithms for Real-World Assumptions

1. Project Overview

The classic TSP asks: given a set of cities and distances between every pair of cities, find the shortest possible tour that visits each city exactly once and returns to the starting city.

At first glance, the classic TSP seems well-defined. However, the classic version depends on many assumptions, such as fixed distances, complete information availability, one salesperson, equal city importance, and a single objective of minimizing total distance.

All teams begin with the same classic problem: the Traveling Salesperson Problem, or TSP.

Your team's task is to identify a realistic use case for which at least one assumption of the classic TSP that does not hold. Based on the modified problem scenario, redesign the algorithm, data structure or both and justify why the redesign is necessary.

The goal is not to find a mathematically optimal solution. The project evaluates the team's ability to redesign the underlying data structure and/or algorithms to support the new requirements and evaluate the trade-offs introduced thereby.

2. Core Learning Goals

1. Understand the classic Traveling Salesperson Problem and the underlying assumptions.
2. Translate the real-world motivation to a new computational problem.
3. Redesign a data structure, an algorithm or both for the new problem setting.
4. Evaluate the redesigned approaches.
5. Use AI tools responsibly while still demonstrating original reasoning.

3. Requirements

Each team must:

1. Define a new use-case of the TSP algorithm.
2. Define a corresponding TSP variant.
3. Implement and evaluate at least two redesigned approaches. The two approaches must differ meaningfully in their data structure design, algorithm design, or both. Different parameter settings of the same implementation do not constitute distinct approaches.

You could explore more than two approaches but choose the two most interesting ones to submit.

Examples of Redesign

- Replacing the static adjacency matrix with a dynamic graph structure.
- Replacing the Nearest Neighbor selection logic with a multi-criteria decision strategy.
- Introducing a new graph representation to support time-varying edges.

Classic TSP Formulation

- Input: A set of n cities and a distance or cost associated with every pair of cities.
- Output: A tour that starts from a city, visits every city exactly once, returns to the starting city and minimizes total cost.

The classic representation is usually a complete weighted graph:

- Each city is a vertex.
- Each possible trip between two cities is an edge.
- Each edge has a weight representing distance or cost.

Common Implementation: Nearest Neighbor TSP

All teams can consider nearest neighbor TSP as a baseline.

Nearest neighbor TSP is useful because it is easy to understand, connects naturally to data structures, is not optimal, and can be modified for many TSP variants.

```
NearestNeighborTSP(cities, distance):
    route = []
    start = cities[0]
    current = start
    unvisited = set of all cities except start

    add start to route

    while unvisited is not empty:
        next_city = city in unvisited with smallest distance(current, city)
        add next_city to route
        remove next_city from unvisited
        current = next_city

    add start to route
    return route
```

4. Choosing the Real-World Use Case

Teams are encouraged to use the following workflow:
Use case → why the Classic TSP is insufficient → new TSP variant → Design Approach A → Design Approach B → Experimental Comparison → Trade-off analysis

Some examples of change in classic key assumptions and possible redesigns are given below:

Classic Key Assumption	Possible Change	Possible Redesign
Distances are fixed	Weather considerations can influence distance	Time-dependent edge weights
Single goal - minimize distance	The objective may include other criteria (e.g. time)	Multi-criteria city-selection algorithm
There is only one salesperson	Multiple salespersons	Clustering algorithm before route generation
All cities are treated equally.	Some locations have greater importance than others	Priority queue or ranking structure

These are just examples. Teams may propose another assumption, provided the scenario is realistic and the redesign is non-trivial.

5. Dataset

The Geonames (https://www.geonames.org/) dataset provided (CS201_2026_project.zip containing the data file MY.txt) must be used as the primary source of data. It contains attributes such as GeoNameID, Name (name of the geographical point), Latitude, Longitude, Feature Class, Population and other geographic information.

There is no minimum dataset size to be used, but the chosen size must be justified. You may augment the dataset with additional attributes relevant to your use case. Any additional attributes must be clearly documented. Teams should **briefly describe** the following:

- Records selected and columns used from the dataset.
- Justification for selection of the partial or augmented dataset.
- Any filtering or preprocessing performed.
- Any supplementary data or derived attributes added.

6. Deadlines and deliverables

Deadline:

- Week 12 Sunday, 11pm: Submission of deliverables via eLearn
- Week 13: Submission of Peer Evaluation via eLearn in class

Deliverables:

(a) Source Codes

The implementation may be coded using Java or Python or any other approved language. You are allowed to use other 3rd party libraries but do include them in the submission along with a readme file detailing instructions to run the source codes.

- The submission must be in the format G<X>T<X>.zip, which:
  - Contains ALL source codes required to reproduce the experiments discussed in the presentation
  - Includes the data files in the format used.
  - Has a proper folder structure for each experiment, labeled meaningfully
  - Includes a README describing the folder structure and instructions to run the code.
- Multiple submissions are allowed but only the most recent submission will be retained.

(b) Presentation slides (described in the next section)

7. Project Presentation

You must cover the following points in your presentation:

a) Motivation and Use Case

- Motivation for real-life scenario.
- Explanation of why the classic TSP does not fully address the chosen use case.

b) New TSP Variant

- New input, output, and constraints.

c) Redesigned Data Structure(s) and/or Algorithm

- Revised Data Structure(s). Alternate structures considered, if any.
- Pseudocode of revised algorithm.
- Experimental Results
- Complexity analysis, Tradeoffs and limitations.
- Interesting findings

d) AI Usage

- Acknowledgement of AI Tools used. Read Section 11 for detailed AI Usage Policy.

We expect all members of the team to contribute during presentation. Questions could be directed towards any member of the team.

8. Grading Rubrics

The project will account for 20% of the overall grade for the course.

Your project is graded on the following components:

- **Interestingness of Problem (10%)**
  - Significance and realism of use case. Be creative!
- **Experimentation (30%)**
  - Data Structures and Algorithms Design
  - Complexity Analysis
  - Experiments & Evaluation
- **Presentation (40%)**
  - Clarity of presentation
  - Efforts reflecting thoughtful planning in presentation.
  - Responses during Q&A
- **Critique of another Team's Design / Algorithm (10%)**
  - Depth and relevance of questions raised.
- **X-factor (10%)**
  - What makes the project stand out from the others.
  - Justify your own X-factors in your presentation. Some good examples of good factor include novel problem formulation, insightful evaluation methodology, innovative data structures. Implementing a fancy UI does not qualify as a X-factor.

9. Peer Evaluation

(a) Each group will perform evaluation of the other teams during the Week 13 project presentations. Each team is expected to pose questions to one presenting group (assigned by the faculty team in Week 13).

(b) Each group member will perform self-evaluation and peer evaluation of their group members that will influence the individual class participation component.

Peer evaluation is mandatory, and non-submissions will be penalized. If there are issues within the group, please alert the teaching team by Week 10 (rather than waiting until final peer evaluation).

10. Project Clarification

All project clarifications must be made through the Course's Teams Channel "Project Clarification Forum".

11. AI Use Policy

Students may use AI tools for brainstorming, experiment design, sample data generation, coding, debugging, and presentation preparation. But you must acknowledge the use, and you remain accountable for every design decision, code and deliverable.

11.1 GenAI Usage approach

GenAI may accelerate your work; it does not replace your responsibility to understand it. The learning outcome is to explain, justify and defend the proposed algorithm and solution.

11.2 Conditions and Boundaries

- **Accountability** — you must be able to explain, justify, and modify any part of your submission on request.
- **No fabrication** — no fake screenshots, logs, test results, or demo evidence.
- **Closed components** — GenAI cannot be used during the live presentation/demo to address queries raised during the presentation.
- Inability to explain your own work will reduce your mark, even if your solution works correctly.

11.3 Acknowledgement of Use

All GenAI use must be acknowledged in your project report submission, stating:

- tools used
- how / where it was used
- parts of the project original
- Include 3 representative AI prompts that helped you design, implement, debug, or improve your project

11.4 Consequences of unpermitted or undeclared use

Non-acknowledgement, fabrication of evidence, disclosure of credentials/confidential data, inability to defend submitted work, or prohibited use during closed components will be reviewed and may be handled under the SMU Code of Academic Integrity.